



AUDIT REPORT

VAPT & OPSEC

August 2026

For



Hashlock Market

Table of Content

Report Overview	05
1. Executive Summary	06
2. Scope and Methodology	08
2.1 Scope	08
2.2 Methodology	08
2.3 Classification Criteria	09
Risk Assessment Summary	10
Number of Issues per Severity	10
Summary of Issues	11
3. Detailed Findings	14
3.1 High Severity	14
1. TRON Funding Handler Missing Timelock Verification	14
2. Server-Side Request Forgery via Webhook Delivery	16
3.2 Medium Severity	17
3. Race Condition in Agreement Acceptance Creates Duplicate Swaps	17
4. WebSocket Quote Submission Bypasses Rate Limiting	18
5. BTC Funding Deduplication Allows Dust-Poison Denial of Service	19
6. Non-Unique Hashlock Index Enables Cross-Swap Event Misrouting	20
7. Shared HTLC Pool Reentrancy Over-Credit	21
8. TRON Watcher Processes Unconfirmed Events Without Finality Guard	22
9. Keeper Auto-Claim Has No Retry on Transient Failure	23
10. Privy Login Rebinds Account Without Independent Ownership Proof	24

11. Bull Board Dashboard Mounted Without Authentication	25
3.3 Low Severity	26
12. Derive-Addresses Endpoint Accepts Replayable Signatures	26
13. Swap Address Mutable After Funding (No Status Guard)	27
14. WebSocket Endpoints Lack Connection Limits and Idle Timeout	28
15. Verbose Database Error Messages Reflected to Clients	29
16. Domain Email Spoofable (DMARC Not Enforced)	30
17. Protocol Fee Not Verified on EVM/TRON Funding	31
18. Non-BTC Swap Pairs Stall in State Machine	32
19. CI/CD Actions Pinned to Mutable Tags With Full Secret Access	33
20. Dependabot Configuration Targets Non-Existent Paths	34
21. No Dependency Audit or Secret Scan in CI Pipeline	35
22. Runtime Environment File World-Readable With Keeper Keys	36
23. Application Sets No Security Headers (Edge Proxy Compensates)	37
24. EVM Factory Does Not Measure Received Token Amount	38
25. JWT Secret Strength Check Only Enforced in Production	39
26. No CAA DNS Record Published	40
27. DNSSEC Not Enabled	41
28. Session Token and Swap Preimage in localStorage Under Weak CSP	42
29. Keeper Dispatches on Literal Chain Name	43
30. Session Tokens Non-Revocable (7-Day Stateless JWT)	44
31. Authentication Middleware Re-Creates Deleted Users From Token	45
32. WebSocket Subscription Guard Race Allows Duplicate Listeners	46
33. Monolithic CI Secret Defeats Per-Value Log Masking	47

34. Container Images Deployed via Mutable Tag Without Verification	48
35. Telegram Mini App Served Without Content Security Policy	49
■ 3.4 Informational	50
Swap Preimage Persistence on Non-Settlement Paths	50
MCP Server Surface on Production	50
Telegram Bot-Bridge Authentication	50
Tron Transaction Builder Proxy	50
4. Operational Security (OPSEC) Assessment	51
4.1 External Footprint	51
4.2 Email and Anti-Phishing	51
4.3 TLS, PKI, and DNS	52
4.4 Client-Side Security	52
4.5 CI/CD and Supply Chain	52
4.6 Secrets and Key Management	52
4.7 Monitoring and Incident Response	53
5. Authenticated Penetration Test Results	54
6. Verified Non-Issues	56
Closing Summary & Disclaimer	57

Report Overview

Project	HASHLOCK MARKETS
Report	Security Assessment Report
Assessment	Penetration Testing & Operational Security Review
Version	1.0
Date	August 2026
Testing Type	VAPT + OPSEC + Source Code Review
Testing Method	Non-destructive, Authorized

1. Executive Summary

This report presents the findings from a comprehensive security assessment of Hashlock Markets, a non-custodial atomic peer-to-peer cross-chain swap platform supporting BTC, EVM, and TRON chains via sealed Request-for-Quote (RFQ) and Hash Time-Locked Contracts (HTLC). The assessment covered both application-level penetration testing and operational security review.

The engagement was conducted as a re-audit, building on an initial assessment performed on 13 August 2026. Six independent domain-specialist reviewers conducted parallel deep source review across the settlement layer, authentication system, API routes, WebSocket endpoints, CI/CD infrastructure, and frontend clients. Every finding was then subjected to adversarial verification by independent reviewers tasked with refuting it. The final classifications reflect this multi-pass scrutiny.

The non-custodial core invariant holds: the keeper service can only claim or refund escrowed funds and cannot redirect principal. On-chain recipients are immutable at funding time. No unauthenticated fund theft, no authentication bypass, no cross-user data leakage, and no frontend cross-site scripting sinks were found in the audited codebase.

However, the assessment identified 35 genuine security findings. Two are rated High severity: one enables theft through a missing timelock verification on the TRON settlement path, and the other is a server-side request forgery through the webhook delivery system that reaches the internal network. Seven findings are rated Medium, including a race condition that creates duplicate swaps (confirmed live), a rate-limit bypass on the WebSocket quote submission path, and several settlement logic defects. The remaining findings address hardening gaps in the CI/CD pipeline, DNS posture, session management, and client-side secret storage.

Key Metrics

Metric	Value
Total Findings	35
High Severity	2
Medium Severity	7
Low Severity	22
Informational	4
Currently Exploitable	14
Latent (conditional risk)	18
Authenticated Test Vectors Run	20+



Metric	Value
Authorization Bypasses Found	0

Important: the production deployment at hashlock.markets runs a different, older Express/Next.js codebase that was not part of this audit. The audited Hono rebuild is currently deployed on the development tier (dev.hashlock.markets / api-dev.hashlock.markets). All findings apply to the audit branch. A separate assessment should be scoped for the production application before the rebuild is promoted.

2. Scope and Methodology

2.1 Scope

Category	Details
Application	Hashlock Markets - Non-custodial atomic cross-chain swap platform
Code Scope	packages/api (Hono backend, BullMQ workers, chain watchers, keeper), packages/settlement-core (client signing library), packages/web (Next.js frontend), packages/tma (Telegram Mini App), CI/CD and deploy configuration. Branch: audit
Host Scope	hashlock.markets, dev.hashlock.markets, api-dev.hashlock.markets, subdomains
Out of Scope	Smart contracts (per client direction), production Express/Next.js codebase
Technology Stack	Hono (HTTP framework), DrizzleORM (PostgreSQL), BullMQ (job queues), Redis, Caddy (reverse proxy), SIWE/TRON/BIP-322/Privy/Telegram auth, HS256 JWT sessions, SHA-256 hashed API keys
Testing Type	White-box source review + authenticated black-box penetration testing + OPSEC review

2.2 Methodology

The original assessment (13 August) involved manual crown-jewel source review followed by sliced multi-agent scanning across six trust-boundary domains, live non-destructive verification, and an authenticated penetration test using three throwaway accounts created through the real SIWE flow.

The re-audit (26 August) employed a structured three-phase approach:

Phase 1: Deep Review: Six domain specialists conducted independent deep source review covering settlement/HTLC logic, authentication/identity, API routes/webhooks, WebSocket/real-time, CI/CD OPSEC, and frontend/client security.

Phase 2: Adversarial Verification: Each finding was independently challenged by reviewers explicitly tasked with refutation. Findings that survived this adversarial pass with concrete source evidence were retained; those refuted with mitigating evidence were reclassified or removed.

Phase 3: Synthesis: A final synthesis pass reconciled all reviews and verdicts, classifying each finding and resolving disagreements between reviewers.

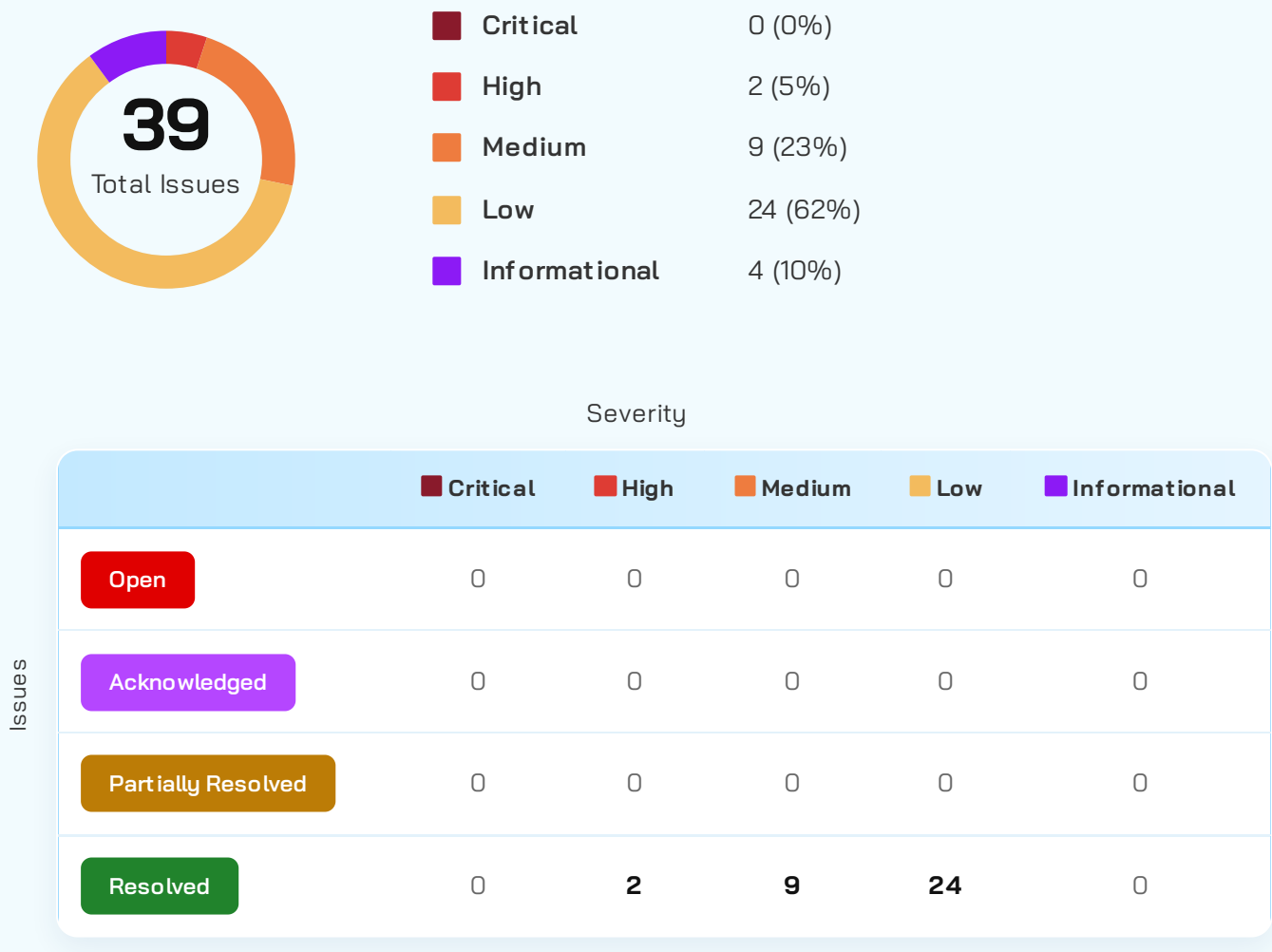
2.3 Classification Criteria

Classification	Definition
Currently Exploitable	The vulnerability can be exploited today with a concrete attack path and evidence.
Latent Risk	A code defect exists but is not exploitable under current conditions. It becomes exploitable if specific conditions change (new token onboarding, configuration change, infrastructure change, etc.).

Findings classified as business decisions (intentional design choices) or false positives (refuted during adversarial verification) have been excluded from this report. Three false positives were identified and removed: cross-site WebSocket hijacking (message-based auth prevents exploitation), maker-feed address case normalization (consistently applied on both sides), and development Docker Compose port exposure (production compose correctly uses internal-only networking).

Risk Assessment Summary

Number of Issues per Severity



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	TRON Funding Handler Missing Timelock Verification	High	Resolved
2	Server-Side Request Forgery via Webhook Delivery	High	Resolved
3	Race Condition in Agreement Acceptance Creates Duplicate Swaps	Medium	Resolved
4	WebSocket Quote Submission Bypasses Rate Limiting	Medium	Resolved
5	BTC Funding Deduplication Allows Dust-Poison Denial of Service	Medium	Resolved
6	Non-Unique Hashlock Index Enables Cross-Swap Event Misrouting	Medium	Resolved
7	Shared HTLC Pool Reentrancy Over-Credit	Medium	Resolved
8	TRON Watcher Processes Unconfirmed Events Without Finality Guard	Medium	Resolved
9	Keeper Auto-Claim Has No Retry on Transient Failure	Medium	Resolved
10	Privy Login Rebinds Account Without Independent Ownership Proof	Medium	Resolved
11	Bull Board Dashboard Mounted Without Authentication	Medium	Resolved
12	Derive-Addresses Endpoint Accepts Replayable Signatures	Low	Resolved
13	Swap Address Mutable After Funding (No Status Guard)	Low	Resolved
14	WebSocket Endpoints Lack Connection Limits and Idle Timeout	Low	Resolved

Issue No.	Issue Title	Severity	Status
15	Verbose Database Error Messages Reflected to Clients	Low	Resolved
16	Domain Email Spoofable (DMARC Not Enforced)	Low	Resolved
17	Protocol Fee Not Verified on EVM/TRON Funding	Low	Resolved
18	Non-BTC Swap Pairs Stall in State Machine	Low	Resolved
19	CI/CD Actions Pinned to Mutable Tags With Full Secret Access	Low	Resolved
20	Dependabot Configuration Targets Non-Existent Paths	Low	Resolved
21	No Dependency Audit or Secret Scan in CI Pipeline	Low	Resolved
22	Runtime Environment File World-Readable With Keeper Keys	Low	Resolved
23	Application Sets No Security Headers (Edge Proxy Compensates)	Low	Resolved
24	EVM Factory Does Not Measure Received Token Amount	Low	Resolved
25	JWT Secret Strength Check Only Enforced in Production	Low	Resolved
26	No CAA DNS Record Published	Low	Resolved
27	DNSSEC Not Enabled	Low	Resolved
28	Session Token and Swap Preimage in localStorage Under Weak CSP	Low	Resolved
29	Keeper Dispatches on Literal Chain Name	Low	Resolved

Issue No.	Issue Title	Severity	Status
30	Session Tokens Non-Revocable (7-Day Stateless JWT)	Low	Resolved
31	Authentication Middleware Re-Creates Deleted Users From Token	Low	Resolved
32	WebSocket Subscription Guard Race Allows Duplicate Listeners	Low	Resolved
33	Monolithic CI Secret Defeats Per-Value Log Masking	Low	Resolved
34	Container Images Deployed via Mutable Tag Without Verification	Low	Resolved
35	Telegram Mini App Served Without Content Security Policy	Low	Resolved
–	Swap Preimage Persistence on Non-Settlement Paths	Informational	–
–	MCP Server Surface on Production	Informational	–
–	Telegram Bot-Bridge Authentication	Informational	–
–	Tron Transaction Builder Proxy	Informational	–

3. Detailed Findings

3.1 High Severity

1. TRON Funding Handler Missing Timelock Verification

Resolved**Severity:** High | **Classification:** Currently Exploitable (CWE-345)

Affected Components

- settlement/service.ts lines 265-307 (onTronFunded)
- chains/tron/watcher.ts lines 4-10, 71-78 (TronFunded interface and event extraction)
- settlement/watchers.ts line 82 (wiring to service)
- Contrast: chains/evm/watcher.ts line 86 and settlement/service.ts lines 137-141 (EVM correctly verifies timelock)

Description

The TRON settlement handler (onTronFunded) validates only the recipient address and funding amount against the agreed swap terms. It does not verify the on-chain timelock value. The TRON Funded event emits the timelock at event result index 5, but the watcher extracts only indices 0, 2, 3, and 4 (swapId, recipient, amount, hashlock). The timelock is never read, never passed through the wiring layer, and the service function has no timelock parameter in its type signature.

By contrast, the EVM handler (onEvmSwapFunded) includes timelock in its mismatch check and rejects any funded event where the on-chain timelock does not match the agreed terms. The TRON path silently skips this critical invariant check.

Because SharedHTLC.fund() accepts a caller-chosen timelock and the TRON-to-EVM pair is creatable (both chains have timelockSeconds=14400, and the tie-break logic assigns TRON as the long/initiator leg), an attacker can fund the TRON leg with an inverted (shorter) timelock. This breaks the asymmetric-timelock core invariant: the attacker can refund their TRON leg early AND claim the counterparty EVM leg using the revealed secret, keeping both sides of the swap.

Impact

Theft of both swap legs. The attacker profits the full value of both the TRON and EVM escrow amounts. Urgency is currently tempered because TRON settlement is configuration-gated (testnet only, pre-Phase-8), but the code path is fully implemented and the sibling EVM check is already live. This is a code defect to fix before TRON settlement goes live, not a hypothetical.

1. TRON Funding Handler Missing Timelock Verification *(continued)*

Remediation

Parse the timelock from the TRON Funded event at index 5 in chains/tron/watcher.ts. Thread it through the watchers.ts wiring layer. In onTronFunded, reject when `Number(timelock) !== legTimelockSec(swap, leg)`, exactly as the EVM handler already does. The `legTimelockSec` function is already chain-generic. This is a straightforward fix.

Status

Resolved

2. Server-Side Request Forgery via Webhook Delivery

Resolved

Severity: High | **Classification:** Currently Exploitable (CWE-918)

Affected Components

- routes/v1.ts line 50 (`z.string().url()` format-only validation at registration)
- webhooks/worker.ts line 25 (`fetch(w.url)` with no SSRF guard)
- webhooks/worker.ts line 40 (`lastStatus` persisted as status oracle)
- routes/v1.ts lines 66-72 (`POST /v1/webhooks/:id/ping` for on-demand triggering)

Description

The webhook registration endpoint validates the URL using only `z.string().url()`, which enforces URL syntax but does not restrict the host, scheme, or IP range. The URL is stored verbatim. The BullMQ delivery worker fetches the stored URL from within the internal Docker network with no restrictions:

- No private, loopback, link-local, or cloud metadata IP blocklist
- No `redirect:manual` setting, so the HTTP client follows up to 20 redirect hops. This means even a registration-time host allowlist is bypassable by pointing the webhook at an allowed external host that 302-redirects to `http://169.254.169.254/` or `http://127.0.0.1:6379`
- `POST /v1/webhooks/:id/ping` triggers delivery on demand, making this an interactive internal port scanner
- The HTTP response status code is persisted in `lastStatus` and returned via `GET /v1/webhooks`, providing a reachability and port-status oracle for internal services

No `requireScope` middleware is applied to webhook routes, so even the lowest-privilege read-only API key can register and trigger webhook deliveries. A whole-tree grep confirms no SSRF or DNS guard exists anywhere in the codebase.

Impact

Blind SSRF with an internal-port status oracle. An attacker can map internal services (Redis on port 6379, Postgres on port 5432, cloud metadata at 169.254.169.254, internal admin panels), and can reach state-changing internal endpoints via crafted POST bodies. The authentication gate is any API key, which is self-service via Privy signup.

Remediation

The fix must operate at the egress layer, not only at registration time. Resolve the target host and reject private, loopback, link-local, and metadata IP ranges at both registration and delivery time. Pin the resolved IP to defeat DNS-rebind TOCTOU attacks. Set `redirect:manual` or use a validating dispatcher. Restrict the scheme to https only. Consider fronting outbound webhook traffic through a dedicated egress proxy with an allowlist. Apply `requireScope` to webhook routes.

Status

Resolved

3.2 Medium Severity

3. Race Condition in Agreement Acceptance Creates Duplicate Swaps

Resolved**Severity:** Medium | **Classification:** Currently Exploitable (CWE-362, CWE-367) - Confirmed Live

Affected Components

- rfq/service.ts lines 610-649 (acceptTerms check-then-act sequence)
- rfq/service.ts lines 703-738 (createSwapFromAgreement INSERT)
- db/schema.ts line 286 (swaps.threadId is a plain column, no unique constraint)
- db/schema.ts line 350 (swaps_thread_idx is a non-unique index)

Description

The acceptTerms function performs a check-then-act sequence without any concurrency control. It reads the thread, checks that status is "open", writes the acceptance flag via an unconditional UPDATE, re-reads the thread, and if both parties have accepted, calls createSwapFromAgreement which INSERTs a new swap row. The status only flips to "agreed" after the insert completes.

There is no database transaction wrapping this sequence, no SELECT FOR UPDATE row lock, and no unique constraint on swaps.threadId. The migration SQL confirms the index is CREATE INDEX, not CREATE UNIQUE INDEX. The idempotency middleware only engages on a voluntarily-sent header and is per-user; the route itself has no idempotency.

This was confirmed live: 15 concurrent acceptance requests for one thread produced 4 separate "agreed" swap rows, all sharing the same threadId, hashlock, and amounts. Reproduced in 2 out of 3 runs.

Impact

One economic agreement becomes multiple independently-settleable swaps. All duplicates share a single hashlock, which chains into finding 6 (cross-swap event misrouting). Revealing the preimage to settle one swap exposes every other funded duplicate. This also double-books keeper jobs, fees, and notifications. The on-chain HTLC claims only once, so this is not direct principal theft, but it corrupts settlement bookkeeping and creates an unpredictable state.

Remediation

Wrap the accept-and-create sequence in a single database transaction. Use a conditional UPDATE with a WHERE status = "open" clause and proceed only if the row was actually updated. Add a partial unique index on swaps.threadId for non-terminal statuses to prevent duplicate swap creation at the database level.

Status

Resolved

4. WebSocket Quote Submission Bypasses Rate Limiting

Resolved

Severity: **Medium** | **Classification:** Currently Exploitable (CWE-770, CWE-799)

Affected Components

- ws.ts lines 76-83, 138-139 (maker WS quote submission handler)
- index.ts lines 50-51, 63 (HTTP rate limiter applied to REST routes only)
- routes/v1.ts line 117 (REST quote endpoint with rate limit and idempotency)

Description

A maker authenticated on the /v1/ws WebSocket endpoint can submit quotes by sending {quote: {rfqId, quoteAmount}} frames. The equivalent REST endpoint (POST /v1/quotes) is protected by the per-API-key rate limiter (300 requests per minute) and idempotency middleware. However, WebSocket frames do not traverse the HTTP middleware chain; those middleware functions execute only once during the upgrade handshake.

As a result, WebSocket quote submissions are entirely unthrottled and non-idempotent.

Impact

Each postQuote call inserts a messages row (unbounded database growth), sends a Telegram direct message to the RFQ creator (notification spam and targeted harassment), and enqueues a quote.created webhook event (outbound amplification). A single maker key on one WebSocket connection can flood an arbitrary RFQ creator and balloon the messages table. The assertCanRespond authorization check is still enforced, so this cannot be used for cross-user access, but it is an authenticated abuse and resource-exhaustion vector exploitable today.

Remediation

Apply the same rate limit and idempotency controls inside the WebSocket quote handler. Track per-key quote submission frequency at the application layer, independent of the HTTP middleware.

Status

Resolved

5. BTC Funding Deduplication Allows Dust-Poison Denial of Service

Resolved

Severity: **Medium** | **Classification:** Currently Exploitable (CWE-841)

Affected Components

- chains/bitcoin/watcher.ts lines 12, 62-64 (FUNDED_KEY, redis.sadd keyed on bare address)
- chains/bitcoin/watcher.ts lines 69-70 (spend path correctly keyed on address:txid)
- settlement/service.ts lines 189-193 (onBtcFunded value validation, downstream of dedup)

Description

The BTC funding watcher deduplicates funding events using redis.sadd keyed on the bare P2WSH address. This dedup check fires as soon as any confirmed output pays the address, before the settlement service validates the funding amount. SADD is one-shot: once the address is marked as seen, subsequent calls return 0 and processing stops.

A 1-satoshi dust output sent to the known P2WSH address permanently poisons that address in the dedup set. When the legitimate full-value funding transaction arrives, SADD returns 0 and it is never processed. There is no SREM, TTL, or reconciliation mechanism. The counterparty knows the deterministic address at the "agreed" status (before funding), providing a clean grieving window.

Notably, the spend path correctly keys on address:txid, proving this is an oversight in the funded path.

Impact

No fund loss (the timelock refund preserves principal), but a reliable liveness denial-of-service that permanently stalls any targeted swap leg.

Remediation

Key the funding dedup on address:txid instead of bare address. Mark the address as seen only after the value and fee checks pass in onBtcFunded.

Status

Resolved

6. Non-Unique Hashlock Index Enables Cross-Swap Event Misrouting

Resolved**Severity:** Medium | **Classification:** Currently Exploitable (CWE-345)

Affected Components

- db/schema.ts line 349 (swaps_hashlock_idx is a plain, non-unique index)
- settlement/service.ts lines 67-70 (findByHashlock uses .limit(1) with no ORDER BY)
- rfq/service.ts lines 628, 703-712 (hashlock from initiator, no uniqueness enforcement)

Description

The swaps table indexes the hashlock column with a plain (non-unique) index, confirmed by both the schema definition and the migration SQL. The findByHashlock function queries using .limit(1) with no ORDER BY clause, making the result non-deterministic when multiple rows share the same hashlock. The initiator supplies the hashlock with no server-side uniqueness enforcement.

An attacker can create a swap reusing a victim's in-flight hashlock. When the victim's legitimate funded event arrives, findByHashlock may return the attacker's row instead. The terms-binding mismatch check rejects it (fail-closed), the event is silently dropped by the durable cursor (no replay), and the victim's swap stalls until timelock refund.

Impact

The fail-closed terms-binding prevents this from becoming theft, capping impact at probabilistic griefing and denial of service. This compounds finding 3 (duplicate swaps share hashlocks).

Remediation

Add a unique constraint on the hashlock column for active (non-terminal) swaps. Alternatively, match funding events by the immutable clone address or on-chain swap ID, reconciling all rows that share a hashlock.

Status

Resolved

7. Shared HTLC Pool Reentrancy Over-Credit

Resolved

Severity: **Medium** | **Classification:** Latent Risk (CWE-841) - Foundry PoC Exists

Affected Components

- packages/contracts/src/tron/SharedHTLC.sol lines 77-111 (fund() with balance-delta accounting)
- packages/contracts/test/AuditPoC_SharedReentrancy.t.sol (working PoC)

Description

The SharedHTLC contract (used for TRON settlement) accounts for funded amounts using a balance-delta pattern: it snapshots the token balance before the external transferFrom call, then credits the difference. The fund function has no reentrancy guard (no nonReentrant modifier). A callback-capable TRC-20 token could re-enter fund() with a different swap ID during the outer call's transferFrom, over-crediting the outer swap and draining a co-resident escrow.

A Foundry proof-of-concept with both positive and negative controls demonstrates this deterministically.

This is not exploitable today because the only onboarded token is standard USDT-TRC20, which has no transferFrom callback. The EVM path is structurally immune because it uses a clone-per-swap architecture with immutable parameters.

Impact

Becomes a direct pool-drain vulnerability if any hook-bearing or ERC-777-style token is added to the shared TRON pool. The asset registry is generic and operator-controlled, with no hook detection logic.

Remediation

Add a nonReentrant modifier to fund(), claim(), and refund() on SharedHTLC before onboarding any non-USDT TRC-20 token. Consider adding a token-capability check to the asset onboarding process.

Status

Resolved

8. TRON Watcher Processes Unconfirmed Events Without Finality Guard

Resolved**Severity:** Medium | **Classification:** Latent Risk

Affected Components

- chains/tron/watcher.ts lines 49-89 (no confirmation depth, no only_confirmed filter)
- chains/evm/watcher.ts lines 48-60, 74 (CONFIRMATIONS=5 with explicit reorg handling)
- chains/bitcoin/watcher.ts lines 9, 55-56 (BTC_CONFIRMATIONS buried() gate)

Description

The TRON watcher processes Funded, Claimed, and Refunded events as soon as TronGrid returns them, gated only by a timestamp cursor and a Redis dedup set. The TronGrid query URL omits the only_confirmed=true parameter, and there is no confirmation-depth or super-representative finality wait.

This directly contradicts the EVM watcher (which uses CONFIRMATIONS=5 with an explicit reorg comment) and the BTC watcher (which gates on a buried() confirmation check).

Impact

A TRON reorganization that drops a Funded event after the backend has advanced the swap state can induce the keeper to commit against a leg that no longer exists on-chain. Recovery is possible via timelock refund, but the inconsistency with the EVM/BTC handling indicates the guard was considered necessary.

Remediation

Apply a confirmation-depth or super-representative finality check before firing TRON event handlers, consistent with the existing EVM and BTC watcher implementations.

Status

Resolved

9. Keeper Auto-Claim Has No Retry on Transient Failure

Resolved

Severity: **Medium** | **Classification:** Latent Risk

Affected Components

- queues.ts lines 46-48 (htlc.claim/refund queues with no defaultJobOptions)
- queues.ts lines 50-53 (webhook.deliver with attempts:6 and exponential backoff)
- settlement/service.ts lines 55-59 (relayCounterLeg with stable jobId)

Description

The htlc.claim and htlc.refund BullMQ queues are created with no defaultJobOptions, which means BullMQ defaults to attempts=1 with no backoff. By contrast, the webhook.deliver queue explicitly configures attempts:6 with exponential backoff.

The relayCounterLeg function enqueues the security-critical counter-leg claim with a stable jobId, and the EVM watcher's durable cursor fires each Claimed event exactly once with no replay.

Impact

A single transient failure (RPC timeout, gas estimation error, nonce collision) fails the job permanently with no retry, and the reveal event is not re-processed. The counter-leg auto-claim silently stalls. Not fund loss (the preimage is public on-chain and the counterparty can claim client-side or refund at timelock expiry), but it degrades the advertised non-custodial auto-settlement guarantee.

Remediation

Configure attempts and exponential backoff for the htlc.claim and htlc.refund queues, consistent with the webhook delivery queue. The stable jobId is compatible with BullMQ retries (retries re-run the same job).

Status

Resolved

10. Privy Login Rebinds Account Without Independent Ownership Proof

Resolved**Severity:** Medium | **Classification:** Latent Risk

Affected Components

- repos/users.ts lines 88-95 (findOrCreateByPrivy attaches DID to existing account by evmAddress match)
- repos/users.ts lines 72-81 (fillEmpty overwrites tronAddress/btcAddress/btcPubkey)
- auth/privy.ts lines 54-63 (fetchPrivyWallets accepts any wallet type including external)

Description

The findOrCreateByPrivy function attaches a Privy DID to any existing Hashlock account whose evmAddress matches the address reported by Privy. The fillEmpty helper then overwrites the account's TRON address, BTC address, and BTC public key with Privy-reported values. The fetchPrivyWallets function accepts any wallet with type "wallet", including externally linked wallets, with no Hashlock-side signature challenge.

Security relies entirely on Privy correctly verifying ownership for every wallet entry it returns.

Impact

If any Privy flow (import, prefill, delegated linking, or a future feature) ever allows a user to attach an address they do not control, an attacker could link a victim's EVM address to their own Privy DID, authenticate via POST /api/auth/privy, and receive the victim's entire Hashlock account with TRON/BTC payout addresses repointed to attacker-controlled wallets.

Remediation

Require an independent Hashlock-side signature challenge before binding a Privy-reported address. Change the fillEmpty behavior to only fill empty fields, never overwrite existing TRON/BTC addresses.

Status

Resolved

11. Bull Board Dashboard Mounted Without Authentication

Resolved

Severity: **Medium** | **Classification:** Latent Risk

Affected Components

- index.ts lines 90-99 (Bull Board at /admin/queues with no auth middleware)
- config.ts lines 112-113 (enabled by default when NODE_ENV is not production)

Description

The Bull Board queue management dashboard is mounted at /admin/queues with no authentication middleware. The application middleware stack (CORS, rate limiting) only covers /api and /v1 paths. Bull Board is enabled by default whenever NODE_ENV is not set to "production".

The sole control keeping it private is that Caddy only proxies /api and /v1 paths, a single-layer topology control.

Impact

On any direct container-port exposure (port 3000), any Caddy misconfiguration forwarding non-/api paths, or in any non-production environment, the dashboard exposes all BullMQ job payloads (webhook target URLs, swap and settlement internals) and provides job retry/remove controls with zero authentication.

Remediation

Add authentication middleware to the Bull Board mount point. Disable Bull Board unless explicitly enabled by configuration, regardless of the NODE_ENV value.

Status

Resolved

3.3 Low Severity

12. Derive-Addresses Endpoint Accepts Replayable Signatures

Resolved**Severity:** Low | **Classification:** Currently Exploitable (CWE-294)

Affected Components

- routes/me.ts lines 76-103; chains/derive.ts lines 21-23

Description

Unlike the SIWE authentication flow (which requires a product marker, single-use nonce, and address binding), the /me/derive-addresses endpoint validates only a bare message and signature via ecrecover with no nonce, product marker, domain separation, or replay protection. Any personal_sign signature the victim has ever produced in any context can be replayed to bind the victim's EVM, TRON, and BTC identities to the attacker's account.

Impact

Pre-registration squatting (victim blocked with 409 on later registration) and private-order interception (assertCanRespond treats the linked address as login-proven ownership).

Remediation

Require a fresh single-use nonce and product marker on the derive-addresses endpoint, consistent with the SIWE authentication flow.

Status

Resolved

13. Swap Address Mutable After Funding (No Status Guard)

Resolved

Severity: Low | **Classification:** Currently Exploitable (CWE-841)

Affected Components

- rfq/service.ts lines 754-792 (setSwapAddress unconditional update)

Description

The setSwapAddress function has no swap.status guard and unconditionally overwrites the caller's payout or refund address at any swap status. A party can observe the counterparty's in-flight fund transaction and change their payout address before the watcher processes it, causing the funded event to be flagged as a spoof (terms-mismatch) and the leg to never be recorded.

Impact

Griefing and liveness degradation. No theft (on-chain recipient is fixed at fund).

Remediation

Reject setSwapAddress once status is past "agreed" or once the leg's HTLC address has been recorded.

Status

Resolved

14. WebSocket Endpoints Lack Connection Limits and Idle Timeout

Resolved

Severity: Low | **Classification:** Currently Exploitable (CWE-400)

Affected Components

- ws.ts lines 36, 93, 98; index.ts lines 49-71 (rate limiter throttles velocity, not concurrency)

Description

Neither WebSocket endpoint implements ping/pong heartbeats, idle timeouts, per-IP concurrent connection caps, or maxPayload limits (the ws library defaults to 100 MiB). Each authenticated socket allocates a Redis subscriber connection. The HTTP rate limiter throttles upgrade velocity but not concurrency, allowing an attacker to accumulate and hold open sockets.

Impact

Resource-exhaustion denial of service against a single-replica service. Authenticated floods also exhaust the Redis connection pool.

Remediation

Add ping/pong with an idle timeout, per-IP concurrent connection limits, and a maxPayload cap.

Status

Resolved

15. Verbose Database Error Messages Reflected to Clients

Resolved

Severity: Low | **Classification:** Currently Exploitable (CWE-209)

Affected Components

- routes/rfqqs.ts lines 44-49 (unauthenticated GET /:id); approximately 47 handlers with the same pattern

Description

GET /api/rfqqs/:id has no authentication requirement and forwards the raw path parameter into a UUID column query. A non-UUID value triggers a Postgres "invalid input syntax for type uuid" error that is reflected verbatim in the response body via the (e as Error).message pattern. This pattern is repeated across approximately 47 route handlers. No global error sanitizer (app.onError) is configured.

Impact

Technology stack and database internals disclosure to unauthenticated clients.

Remediation

Validate path parameters (e.g., zod UUID check) before database queries. Add a global app.onError handler that maps unexpected errors to generic messages, whitelisting only typed validation errors.

Status

Resolved

16. Domain Email Spoofable (DMARC Not Enforced)

Resolved

Severity: Low | **Classification:** Currently Exploitable

Affected Components

- DNS records: DMARC p=none, SPF ~all (softfail)

Description

DMARC is set to p=none (monitor only) and SPF uses ~all (softfail), meaning receiving mail servers do not reject or quarantine spoofed mail from hashlock.markets. Anyone can send email with From: team@hashlock.markets and it will largely be delivered. DKIM is present but does not help because an attacker simply omits it.

Impact

Phishing enablement against a financial/crypto brand. The team uses email for business operations (the DMARC rua address confirms this).

Remediation

Move DMARC to p=quarantine and then p=reject with pct=100, adkim=s, aspf=s. Tighten SPF to -all (hardfail).

Status

Resolved

17. Protocol Fee Not Verified on EVM/TRON Funding

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- settlement/service.ts lines 137-141, 286-295 (no fee check); contrast lines 196-210 (BTC enforces fee)

Description

The EVM and TRON funding handlers verify recipient, amount, token, and (for EVM) timelock, but never verify protocol fee payment. Only the BTC handler enforces that the funding transaction pays the treasury address at least the fee amount. The EVM SwapCreated event does not carry a fee field, making verification structurally impossible on the current EVM contract.

Impact

Dormant because PROTOCOL_FEE_BPS and feeBps both default to 0. If fees are enabled, takers whose fee-bearing leg settles on EVM or TRON pay nothing. Revenue loss, not principal theft.

Remediation

Redesign the fee collection mechanism to be verifiable across all chains, or enforce fee collection at a different layer before enabling non-zero fee rates.

Status

Resolved

18. Non-BTC Swap Pairs Stall in State Machine

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- settlement/service.ts lines 41-43 (shortLegFunded only advances from initiator_funded); lines 216-217 (only onBtcFunded sets initiator_funded)

Description

The shortLegFunded function, which advances the swap state machine past the funding phase, only transitions from the initiator_funded status. Only onBtcFunded sets this status. For an EVM-to-TRON swap (creatable via the current pair configuration), both legs fund successfully but the state remains "agreed" and never advances.

Impact

Fail-safe: no theft or fund loss. Funds stay refundable on-chain. But funding notifications, webhooks, and the reveal relay may not fire correctly. Bites when multi-chain expansion enables non-BTC pairs.

Remediation

Generalize the state machine to advance on both legs regardless of which chain is involved.

Status

Resolved

19. CI/CD Actions Pinned to Mutable Tags With Full Secret Access

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- .github/workflows/deploy.yml (all third-party actions pinned to @vN tags)

Description

Every third-party GitHub Action in the deploy workflow is pinned to a mutable version tag, not a commit SHA. The appleboy actions receive the SSH key, and the deploy job renders all runtime secrets (JWT secret, keeper keys, database password, Privy secret) into the .env file in the same job. A compromised or re-tagged upstream action would have access to all production secrets.

Impact

Requires an upstream action compromise, not independently exploitable against the running service.

Remediation

SHA-pin every third-party action. The github-actions Dependabot ecosystem will still propose version bumps.

Status

Resolved

20. Dependabot Configuration Targets Non-Existent Paths

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- .github/dependabot.yml (3 of 5 monitored paths do not exist in this repository)

Description

The dependabot.yml file was copied from another project. It monitors /contracts (should be a Foundry project, not npm), /backend/services/api-gateway (nonexistent), and /web as docker (real path is packages/web). Groups and ignore rules reference packages absent from every package.json. The Docker base images in packages/api, packages/web, and packages/tma receive no CVE-bump PRs.

Impact

An unpatched base-image or Foundry library vulnerability would linger unflagged while the Dependabot dashboard shows "enabled".

Remediation

Retarget Dependabot to the actual package paths and ecosystems in this repository.

Status

Resolved

21. No Dependency Audit or Secret Scan in CI Pipeline

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- .github/workflows/ci.yml (only build/typecheck/lint and forge test)

Description

The CI pipeline runs only build, typecheck, lint, and Forge contract tests. There is no pnpm audit, no OSV-scanner, no gitleaks or trufflehog secret scan, and no Slither static analysis.

Impact

A vulnerable transitive dependency or accidentally committed secret could merge undetected.

Remediation

Add pnpm audit --audit-level=high (or osv-scanner), a secret scan (gitleaks/trufflehog) on pull requests, and Slither on the contracts directory.

Status

Resolved

22. Runtime Environment File World-Readable With Keeper Keys

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- .github/workflows/deploy.yml line 131 (install -m 644); deploy/compose.deploy.yml (env_file reference)

Description

The deploy workflow creates the runtime .env file with mode 644 (world-readable) and appends all secrets from DEPLOY_ENV, including keeper private keys for EVM, TRON, and BTC chains, the JWT secret, the database password, and the Privy application secret. No post-copy chmod is performed on the target host.

Impact

Requires a second local principal on the presumably single-tenant deployment box. Keeper keys are currently testnet-only. Becomes critical with mainnet keys, additional local users, backup/rsync exposure, or container escape.

Remediation

chmod 600 the .env file owned by the deploy user, or move secrets to a Docker secret mount or Vault agent.

Status

Resolved

23. Application Sets No Security Headers (Edge Proxy Compensates)

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- index.ts (no secureHeaders middleware); deploy/Caddyfile lines 6-14 (Caddy sets headers at edge)

Description

The Hono application sets no security headers itself. Caddy provides HSTS, X-Content-Type-Options, X-Frame-Options, and Referrer-Policy at the edge. The residual gap is defense-in-depth only: if the application is ever reached without Caddy, no security headers are present. The API-domain Caddy block also omits X-Frame-Options and CSP (low-value for a JSON API).

The /api path exclusion from Caddy Basic Auth is a deliberate design choice (the Telegram Mini App webview cannot answer a basic-auth prompt). This is not a vulnerability.

Impact

Defense-in-depth hardening gap. No independent exploitability.

Remediation

Add secureHeaders middleware to the Hono application as a fallback layer.

Status

Resolved

24. EVM Factory Does Not Measure Received Token Amount

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- packages/contracts/src/HTLCFactory.sol lines 118-130; contrast SharedHTLC.sol lines 98-104 (TRON measures)

Description

The EVM HTLCFactory._fund function calls safeTransferFrom for the requested amount without measuring the actual received balance. The clone pays out the immutable amount value, so a fee-on-transfer or rebasing token under-funds the clone and both claim and refund revert.

Impact

Self-denial-of-service if a fee-on-transfer token is onboarded. No cross-user theft. Not reachable with native ETH or standard USDT.

Remediation

Measure the received amount using a balance-delta pattern, consistent with the TRON SharedHTLC implementation. Add a capability check to the asset onboarding process.

Status

Resolved

25. JWT Secret Strength Check Only Enforced in Production

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- config.ts lines 5, 11, 84-88 (assertProductionSafe early-returns for non-production)

Description

The JWT secret defaults to the source-public string "dev-secret-change-me" (config.ts line 11). The assertProductionSafe function returns early for any NODE_ENV other than "production" and only rejects the default or short secrets in production mode. NODE_ENV itself defaults to "development".

The live development instance was verified to use a strong secret (a forged token was rejected with 401). The gap is that any non-production deployment that omits NODE_ENV boots with a known HMAC key, enabling universal token forgery.

Impact

Token forgery on any internet-reachable non-production instance where NODE_ENV is not explicitly set to production.

Remediation

Run the anti-default and minimum-length secret check in every environment, not only production.

Status

Resolved

26. No CAA DNS Record Published

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- DNS query: `dig CAA hashlock.markets` returns empty

Description

No Certificate Authority Authorization record is published. Any public certificate authority in the world may issue a certificate for hashlock.markets.

Impact

Defense-in-depth PKI gap. Exploitable only if a CA is compromised or tricked into mis-issuance.

Remediation

Publish CAA 0 issue "letsencrypt.org" with an iodef mailto for alerting on unauthorized issuance attempts.

Status

Resolved

27. DNSSEC Not Enabled

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- DNS query: no DS/DNSKEY records; registrar is Namecheap (supports DNSSEC)

Description

DNS answers are unsigned. Cryptographic protection against cache-poisoning and spoofing is not in place.

Impact

Exploitable only by an on-path or cache-poisoning adversary.

Remediation

Enable DNSSEC via the Namecheap registrar (one-time configuration step).

Status

Resolved

28. Session Token and Swap Preimage in localStorage Under Weak CSP

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- packages/web/lib/api.ts lines 17-24; packages/settlement-core/src/secret.ts lines 20-24; packages/web/next.config.mjs lines 30, 34

Description

The session JWT (hl.token) and the atomic-swap preimage (hl.secret.<threadId>) are stored in browser localStorage. The Content Security Policy allows unsafe-inline, unsafe-eval, and connect-src https: (any HTTPS host).

No XSS sink exists in the current frontend (no dangerouslySetInnerHTML, innerHTML, or eval of user data). React default-escapes all content. This is a latent defense-in-depth weakness. The risk vector is the large wallet dependency tree (wagmi, AppKit, WalletConnect, Privy, MetaMask SDK).

Impact

If any XSS is introduced (e.g., via a compromised transitive dependency), the attacker can exfiltrate the session token and any in-flight swap preimage to any HTTPS host.

Remediation

Move the session token to an HttpOnly, Secure, SameSite=Strict cookie. Store the preimage in memory or IndexedDB with a short lifetime. Tighten CSP toward nonces and strict-dynamic with a connect-src host allowlist.

Status

Resolved

29. Keeper Dispatches on Literal Chain Name

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- settlement/keeper-worker.ts lines 26, 39, 53

Description

The keeper worker dispatches claim and refund operations using exact string equality against "ethereum" and "tron". Any other chain name throws "keeper claim on \${chain} not implemented yet". The codebase is explicitly built to support 20+ networks via configuration.

Impact

The first EVM Layer-2 chain added with keeperAutoClaim enabled permanently strands the counter-leg auto-claim for swaps on that chain. Combined with the no-retry finding (9), this is a permanent failure.

Remediation

Dispatch on chainFamily(chain) (evm/tvm) rather than exact chain name literals.

Status

Resolved

30. Session Tokens Non-Revocable (7-Day Stateless JWT)

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- auth/jwt.ts lines 15-22 (HS256, 7-day expiry, no jti/token-version)

Description

Session JWTs are minted as stateless HS256 tokens with a fixed 7-day expiry and no jti or token-version claim. There is no server-side token store and no denylist. The requireAuth middleware even re-creates a deleted user from the token's address claim. The only revocation lever is rotating JWT_SECRET, which invalidates all active sessions. Revocation is implemented for API keys but not for user sessions.

Impact

A compromised token (via XSS per finding 28, phishing, request-log capture, or shared device) remains valid for up to 7 days and cannot be individually revoked.

Remediation

Add jti and a token-version claim with a server-side denylist, or switch to short-lived access tokens with a refresh token rotation.

Status

Resolved

31. Authentication Middleware Re-Creates Deleted Users From Token

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- auth/middleware.ts lines 14-22; auth/jwt.ts lines 15-31

Description

The resolveUserId function in the authentication middleware resolves the JWT subject to a user row. If the row does not exist, it re-creates the account from the JWT's address and family claims via findOrCreateByEvm or findOrCreateByTron.

Impact

Intended for database-reset resilience. However, it means user deletion cannot revoke a session, and if an address is later re-registered by a different party, an old token could resolve to the new owner. Requires row deletion plus re-registration of the same address.

Remediation

Reject authentication on a missing user row instead of re-creating the account.

Status

Resolved

32. WebSocket Subscription Guard Race Allows Duplicate Listeners

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- ws.ts lines 49-62 (authed flag set after resolveApiKey await); lines 101-124 (subscribed after verifyJwt)

Description

Both WebSocket handlers set their single-shot guard flag after an asynchronous operation (await). Multiple messages dispatched in a single burst can race past the guard before it is set, causing duplicate Redis subscriber listeners to be attached. Each subsequent event is then delivered N times.

Impact

Self-amplification that compounds finding 14 (WebSocket resource exhaustion). No cross-tenant data leakage.

Remediation

Set the guard flag synchronously before the await, or reject additional authentication/subscribe frames.

Status

Resolved

33. Monolithic CI Secret Defeats Per-Value Log Masking

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- .github/workflows/deploy.yml line 132 (DEPLOY_ENV rendered via printf)

Description

All runtime secrets (JWT secret, keeper keys, database password, Privy secret) are packed into a single GitHub Actions secret (DEPLOY_ENV) and rendered with printf. GitHub Actions auto-masking only redacts the full registered secret value, not arbitrary embedded sub-values.

Impact

If any future change surfaces a single sub-value (a debug echo, set -x, or environment interpolation warning), that value prints unredacted in CI logs. The blast radius includes keeper private keys.

Remediation

Register each high-value secret as a separate GitHub Actions secret so that each is independently masked. Alternatively, inject secrets via a secrets manager.

Status

Resolved

34. Container Images Deployed via Mutable Tag Without Verification

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- .github/workflows/deploy.yml lines 93-96, 133-134, 194-196;
deploy/compose.deploy.yml

Description

The forward deploy resolves container images by the mutable :latest tag with no deploy-by-digest, no cosign signature verification, and no provenance attestation check. Docker compose pull accepts whatever currently points at :latest in the GitHub Container Registry.

Impact

A registry compromise or any push that moves :latest is deployed to the host holding keeper keys with no integrity check. Rollback.yml does deploy by explicit SHA.

Remediation

Deploy by immutable image digest. Verify a cosign signature or attestation before docker compose up.

Status

Resolved

35. Telegram Mini App Served Without Content Security Policy

Resolved

Severity: Low | **Classification:** Latent Risk

Affected Components

- `deploy/Caddyfile` lines 20-24, 32-37; `packages/tma/miniapp/index.html`

Description

The Telegram Mini App is a static Vite SPA served by Caddy `file_server`. The Next.js CSP does not apply to it. The Caddy `/mini` path override contains only `frame-ancestors` with no `default-src`, `script-src`, or `connect-src`. The `index.html` file carries no CSP meta tag. Yet the TMA stores the same session JWT and swap preimage in `localStorage`.

Impact

Strictly weaker than the already-weak web CSP. Any XSS or supply-chain compromise would exfiltrate the token and preimage with zero CSP friction. No XSS sink was found in the TMA source.

Remediation

Apply a `script-src` and `connect-src` Content Security Policy to the `/mini` path, consistent with the web CSP.

Status

Resolved

3.4 Informational

The following items are informational observations that do not represent exploitable vulnerabilities but warrant awareness.

Swap Preimage Persistence on Non-Settlement Paths

The `clearSecret` function for the swap preimage (`hl.secret.<threadId>`) is invoked only when a swap reaches the "counterparty_claimed" terminal state. On refund, failure, or expiry paths, and on session disconnect, the preimage lingers indefinitely in `localStorage`. The TMA never clears it. This is not a cryptographic exposure (an unrevealed preimage is useless after refund; a revealed one is already public on-chain), but it is a hygiene gap that widens the window for findings 28 and 35.

MCP Server Surface on Production

The production deployment exposes a Model Context Protocol server at `mcp.hashlock.markets`. The SIWE-Bearer authentication gate is correctly implemented (401 response, spec-compliant OAuth resource metadata, tools/list leaks nothing unauthenticated). Deploying an authenticated MCP surface is an intentional product decision. A dedicated authenticated review of the per-tool authorization and argument validation is recommended before this surface is considered hardened.

Telegram Bot-Bridge Authentication

The `/api/auth/telegram/bot` endpoint issues a Hashlock JWT for any `telegramId` in the request body, gated by a constant-time comparison against a shared `BOT_BRIDGE_SECRET`. This is an intentional trusted-bot design and the `timingSafeEqual` implementation is correct. The concern is blast radius: a single static, universal, non-revocable impersonation key for every Telegram user compounds any secret leak (e.g., the world-readable `.env` file in finding 22). Scoping or rotating the bridge secret is recommended.

Tron Transaction Builder Proxy

The `/me/tron/build` and `/me/tron/broadcast` endpoints allow authenticated users to build unsigned TRON smart contract calls and relay signed transactions. The server never signs anything; the owner field is forced to the caller's own registered `tronAddress`; the broadcast endpoint requires a valid signature. This is a stateless builder/relay for Privy browser users who lack TronWeb. It is not SSRF (the target host is the fixed TronGrid node). This is an intentional design choice and is safe as written.

4. Operational Security (OPSEC) Assessment

This section covers the operational and infrastructure security posture: what an attacker sees from the outside, how the platform is deployed and run, and where the runtime posture creates risk independent of specific code bugs. Detailed findings that overlap with the application assessment are cross-referenced to their finding numbers above.

4.1 External Footprint

Host	IP	Role	Exposed Ports
hashlock.markets / www	207.154.201.207	Production (separate codebase)	22, 80, 443
mcp.hashlock.markets	207.154.201.207	MCP server (SIWE- gated)	443
dev.hashlock.markets	165.245.209.52	Dev app (audited Hono, Caddy)	22, 80
api- dev.hashlock.markets	165.245.209.52	Dev public API	22, 80

Four hosts resolve. No subdomain takeover risk was identified. The www record is a clean CNAME to the apex. Certificate transparency and DNS enumeration surfaced no additional subdomains. The dev box exposes only SSH and Caddy (no direct application ports). Internal services (Postgres, Redis, BullMQ) are correctly isolated behind Docker internal networking in the production compose configuration.

Recommendation: restrict SSH access behind an allowlist, bastion host, or Tailscale instead of exposing it to the internet on both hosts.

4.2 Email and Anti-Phishing

This is the weakest external-facing item. For a financial platform, email spoofing is the cheapest path to phish users and team members.

Current DNS records: SPF is v=spf1include:_spf.google.com ~all (softfail, not hardfail). DKIM is present via Google Workspace. DMARC is v=DMARC1;p=none (monitor only). The combination means receiving mail servers do not reject or quarantine spoofed mail. See finding 16 for remediation steps.

4.3 TLS, PKI, and DNS

TLS certificates are valid Let's Encrypt certificates with correct SANs, auto-renewed by Caddy. HSTS is set with max-age=31536000 and includeSubDomains. Consider adding preload.

No CAA record is published (finding 26). No DNSSEC is enabled (finding 27). The registrar and DNS accounts are top-tier targets controlling MX, ACME validation, and every host. Ensure these accounts have phishing-resistant MFA (hardware keys) and that registrar lock is enabled.

4.4 Client-Side Security

The trader UI and Telegram Mini App store the session JWT and atomic-swap preimage in browser localStorage. The web CSP allows unsafe-inline and unsafe-eval (a documented wallet-stack tradeoff). The TMA has effectively no CSP. No XSS sink exists in the current frontend code (React escapes all content), making these latent defense-in-depth gaps rather than live exposures. The primary risk vector is the large wallet dependency tree. See findings 28 and 35.

4.5 CI/CD and Supply Chain

The deploy workflow pins all third-party GitHub Actions to mutable version tags while handling the SSH key and all production secrets (finding 19). The Dependabot configuration monitors paths copied from another project (finding 20). The CI pipeline has no dependency audit, secret scan, or static analysis gate (finding 21). Container images are deployed via the mutable :latest tag with no digest pinning or signature verification (finding 34). All secrets are packed into one monolithic DEPLOY_ENV variable that defeats per-value log masking (finding 33).

Repository hygiene is good: .gitignore blocks all .env, .pem, .key, .p12, keystore, and secrets paths. No secrets are committed. The .env.example placeholder JWT is length-rejected in production.

4.6 Secrets and Key Management

Keeper hot-wallet keys are the crown-jewel secrets. The project documentation states they should be managed via Vault or KMS, but the deploy path injects them as plaintext environment variables in a world-readable .env file (finding 22). Before mainnet:

- Custody keeper keys in KMS, HSM, or Vault with per-operation signing
- Segregate keeper wallets per chain and keep only working balances hot
- Put contract owner, FEE_ADMIN_ROLE, and TRON SharedHTLC owner behind a multisig
- Verify on-chain that the deployed owner is not a single externally-owned account

The keeper can only claim and refund; it cannot redirect funds (recipients are immutable at fund time). This bounds a key compromise to gas grieving and settling to already-fixed recipients, but a leaked key still allows front-running/settling and burning gas.

4.7 Monitoring and Incident Response

Nothing in the repository evidences runtime security monitoring. For a value-bearing swap platform, the following are recommended:

- Wire the settlement watchers to anomaly alerting (unexpected clones, terms-mismatch events, keeper failures, pool insolvency)
- Ensure a kill-switch exists (`SharedHTLC.freeze()` is available; create an equivalent for the EVM factory and API) with a documented runbook
- Log security-relevant events (auth failures, rate-limit 429 responses, webhook delivery targets) to a central, tamper-evident store with spike alerting
- Document incident response procedures: on-call rotation, key-rotation runbook (keeper, JWT, bot-bridge, Privy secrets), and customer communication plan
- Publish a `/.well-known/security.txt` file (currently returns 404)

5. Authenticated Penetration Test Results

Three throwaway accounts were created through the real SIWE authentication flow using cast wallet. Account A (taker), Account B (maker/initiator), and Account C (non-party). A full swap negotiation was formed between A and B. The following test vectors were executed against the development deployment.

Test Vector	Result
Thread access as non-party C	Blocked ("not a party to this thread")
Swap access as non-party C	Blocked ("not a party to this swap")
Cancel/delete A's RFQ as B or C	Blocked ("not your rfq")
Delete B's address book entry as A or C	No-op (B's address survives, userId-scoped)
List RFQ quotes as non-creator C	Empty array (no cross-user leak)
Mass assignment (inject creatorId/status)	Ignored (RFQ owned by session user)
JWT algorithm:none forgery	Rejected (pinned algorithms: HS256)
Read secretCiphertext pre-reveal	Null (preimage never exposed before reveal)
Reveal wrong 32-byte secret	Rejected ("secret does not match the hashlock")
Input validation (negative amount, 10-year TTL)	Rejected at zod/service layer
Balance check on private order	Actively enforced live
Tron broadcast arbitrary unsigned blob	Rejected ("Transaction is not signed")
Auth rate limit (20 per minute)	Enforced (20x200, then 429)
SIWE nonce replay	Blocked ("nonce invalid or expired", single-use)
WebSocket subscribe to non-owned swap	Blocked ("not a party to this swap")
15 simultaneous acceptTerms requests	Created 4 duplicate swaps (finding 3)

The authorization model held on every test. No object-level access control bypass, no privilege escalation, and no mass-assignment vulnerability was found. The only defect the authenticated pass surfaced was the race condition in finding 3 (a state-machine concurrency issue, not an authorization bypass).

6. Verified Non-Issues

The following areas were explicitly tested and verified to be secure:

Non-custodial invariant: The keeper can only claim and refund. Recipients are immutable at fund time. The keeper cannot redirect funds under any circumstance.

EVM HTLC clone architecture: Immutable CWIA arguments, checks-effects-interactions pattern, settle-once semantics, CREATE2 deterministic addressing, MIN_TIMELOCK floor. Immune to the reentrancy finding that affects the TRON SharedHTLC.

All authentication signature verifiers: SIWE, TRON verifyMessageV2, BIP-322, Telegram HMAC-SHA256, Privy JWKS, JWT pinned-algorithm HS256, API key SHA-256 hashing. All verified correct.

Preimage confidentiality: The swap preimage never transits the pub/sub event system. It is only revealed on-chain when a party claims.

Client-side Bitcoin signature handling: Low-S enforcement plus local verification against the public key and sighash. No malleability or NULLFAIL risk.

Cross-Site Request Forgery: Both WebSocket endpoints use message-based credentials (JWT and API key), not ambient cookies or headers. CSWSH is not exploitable.

SQL injection: DrizzleORM parameterizes all queries. Confirmed by the verbose error finding (15): the bound-parameter error proves injection is not possible.

XFF rate-limit trust: The rate limiter keys on the right-most X-Forwarded-For value, which is safe because the application is reachable only through the Caddy reverse proxy (no direct port exposure).

CORS configuration: /v1 is open-origin with bearer-auth (intentional for API consumers). /api is locked to an explicit allow-list.

Object-level authorization / IDOR: Three-account test matrix found no access control bypass across threads, swaps, RFQs, quotes, address books, or WebSocket subscriptions.

Secrets in repository: .gitignore blocks all sensitive file patterns. No secrets are committed.

Atomic-swap free option: Inherent to HTLC-based atomic swaps, mitigated by the 2-hour minimum timelock gap. Not a bug.

Closing Summary

In this report, we have considered the security of **Hashlock Market**. We performed our audit according to the procedure described above.

The assessment identified a lot of high,medium severity findings, which Hashlock team has resolved.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

August 2026

For



Hashlock Market



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com